# Formalization of Gödel's Incompleteness Theorem in a Guard/Rose Equational System

## 1 Overview

We formalize Gödel's first incompleteness theorem in a system combining two ideas:

- *Guard*'s syntax [2]: binary trees with typed unary and binary functors, serving as both the data domain and the term language;
- *Rose*'s purely equational proof style [3]: the only judgement is an equation $t = u$ between terms, with no propositional connectives, no sequents, and no logical rules — only computation axioms, congruence, substitution, and a tree-induction principle (**Schema F**).

The formalization is in Agda with `--safe --without-K --exact-split`. No postulates.

**Theorem 1** (Gödel I — `godelII.agda`)**.** *For any hypothesis equation $H$:*

$$\mathit{ProofE}(H) \;\to\; \mathit{Consistent}(H) \;\to\; H \vdash G \;\to\; \bot$$

This says: if the proof encoding is correct and the system is consistent, then the Gödel sentence $G$ is not derivable. Here:

- $\mathit{ProofE}(H)$: proof encoding correctness — every derivation from $H$ has a proof tree whose thFunTFor image equals the code of the derived equation (proved constructively by thm14E);
- $\mathit{Consistent}(H)$: no derivation of $O = \mathbf{f}$ from $H$;
- $G$: the Gödel sentence, which (after diagonalization) says "no proof tree encodes a derivation of this sentence."

**Remark 2.** *This is Gödel's* first *incompleteness theorem: $G$ is not provable (if the system is consistent). In Rose's paper [3], this corresponds to Theorem 18. Gödel's* second *incompleteness theorem (*Con *is not provable) would additionally require formalizing the proof of Theorem 1* inside *the object system, corresponding to Rose's Theorems 15–17. This internalization is not carried out here; the formalization proves Gödel I only.*

## 2 The formal system

### Trees

The ground data are finite binary trees:

$$a, b \;::=\; O \;\mid\; \langle a, b\rangle$$

where $O$ is the leaf and $\langle a, b\rangle$ is the node. We define:

$$\mathbf{t} \;=\; O, \qquad \mathbf{f} \;=\; \langle O, O\rangle.$$

Here $\mathbf{t}$ represents equality (or "yes") and $\mathbf{f}$ represents inequality (or "no"). We write $\overline{n}$ for the unary encoding of $n \in \mathbb{N}$: $\overline{0} = O$, $\overline{n+1} = \langle O, \overline{n}\rangle$.

## Terms and functors

Terms are built from trees, variables, and functor applications. We use $t, u, v$ for terms, $a, b$ for trees. The system has countably many variables $x_0, x_1, x_2, \ldots$. There are unary functors ($f, g : \mathrm{Fun}_1$) and binary functors ($h : \mathrm{Fun}_2$).

| | | | |
|---|---|---|---|
| $f, g$ | $::=$ | $\mathrm{I}$ | identity |
| | $\mid$ | $\pi_1$ | first projection |
| | $\mid$ | $\pi_2$ | second projection |
| | $\mid$ | $f \circ g$ | composition |
| | $\mid$ | $h[f, g]$ | binary composition: $t \mapsto h(f(t), g(t))$ |
| | $\mid$ | $\mathrm{R}(z, s)$ | tree recursion with base $z$ and step $s$ |
| | $\mid$ | $\mathrm{K}_c$ | constant $c$ |
| $h$ | $::=$ | $\mathrm{P}$ | pairing: $(t, u) \mapsto \langle t, u\rangle$ |
| | $\mid$ | $\mathrm{C}$ | left projection: $(t, u) \mapsto t$ |
| | $\mid$ | $\mathrm{Lf}(f)$ | lift: $(t, u) \mapsto f(t)$ |
| | $\mid$ | $\mathrm{Ps}(f, h)$ | post-compose: $(t, u) \mapsto f(h(t, u))$ |
| | $\mid$ | $\mathrm{Fn}(h_1, h_2, h)$ | fan: $(t, u) \mapsto h(h_1(t, u),\ h_2(t, u))$ |
| | $\mid$ | $\mathrm{If}$ | conditional |
| | $\mid$ | $\mathrm{Eq}$ | tree equality test |

Terms:

$$t, u \;::=\; O \;\mid\; x_i \;\mid\; f(t) \;\mid\; h(t, u)$$

An *equation* is a pair $t = u$ of terms.

## Embedding trees into terms

Every tree $a$ determines a closed term $\hat{a}$:

$$\hat{O} = O, \qquad \widehat{\langle a, b\rangle} = \langle \hat{a}, \hat{b}\rangle.$$

(In Agda: `reify`.) Conversely, every closed term reduces to $\hat{a}$ for a unique $a$.

## Derivation rules

A derivation is built from the rules below. Every rule may use a hypothesis $H$; we write $H \vdash t = u$.

**Computation axioms.**

| | |
|---|---|
| Identity: | $\mathrm{I}(t) = t$ |
| First projection: | $\pi_1(\langle t, u\rangle) = t$ |
| Second projection: | $\pi_2(\langle t, u\rangle) = u$ |
| Constant: | $\mathrm{C}(t, u) = t$ |
| Composition: | $(f \circ g)(t) = f(g(t))$ |
| Binary composition: | $h[f, g](t) = h(f(t),\ g(t))$ |
| Lift: | $\mathrm{Lf}(f)(t, u) = f(t)$ |
| Post-compose: | $\mathrm{Ps}(f, h)(t, u) = f(h(t, u))$ |
| Fan: | $\mathrm{Fn}(h_1, h_2, h)(t, u) = h(h_1(t, u),\ h_2(t, u))$ |
| Constant functor: | $\mathrm{K}_c(t) = c$ |

**Tree recursion.**

$$\begin{aligned} &\text{Base:} \quad \mathrm{R}(z, s)(O) = z \\ &\text{Step:} \quad \mathrm{R}(z, s)(\langle t, u\rangle) = s\big(\langle t, u\rangle,\ \langle \mathrm{R}(z, s)(t), \mathrm{R}(z, s)(u)\rangle\big) \end{aligned}$$

**Conditional.**

$$\begin{aligned} &\text{True branch:} \quad \mathrm{If}(\mathbf{t},\ \langle t, u\rangle) = t \\ &\text{False branch:} \quad \mathrm{If}(\langle t_1, t_2\rangle,\ \langle u_1, u_2\rangle) = u_2 \end{aligned}$$

More generally, If branches on whether its first argument is $O$ (leaf) or a pair (node).

**Tree equality.**

$$\begin{aligned} &\mathrm{Eq}(O, O) &&= \mathbf{t} \\ &\mathrm{Eq}(O, \langle t, u\rangle) &&= \mathbf{f} \\ &\mathrm{Eq}(\langle t, u\rangle, O) &&= \mathbf{f} \\ &\mathrm{Eq}(\langle t_1, t_2\rangle, \langle u_1, u_2\rangle) &&= \mathrm{If}\big(\mathrm{Eq}(t_1, u_1),\ \langle \mathrm{Eq}(t_2, u_2), \mathbf{f}\rangle\big) \end{aligned}$$

Eq returns **t** for equal trees and **f** for unequal. The last clause says: if the left children differ, return **f** immediately; otherwise, recurse on the right children.

**Structural rules.**

$$\frac{}{t = t}\ (\mathrm{refl}) \qquad \frac{t = u}{u = t}\ (\mathrm{sym}) \qquad \frac{t = u \qquad u = v}{t = v}\ (\mathrm{trans})$$

$$\frac{t = u}{f(t) = f(u)}\ (\mathrm{cong}_1) \qquad \frac{t = u}{h(t, v) = h(u, v)}\ (\mathrm{cong}_L) \qquad \frac{t = u}{h(v, t) = h(v, u)}\ (\mathrm{cong}_R)$$

**Substitution.**

$$\frac{H \vdash t = u}{H \vdash t[c/x_i] = u[c/x_i]}\ (\mathrm{inst})$$

**Hypothesis.**

$$\frac{}{H \vdash H}\ (\mathrm{hyp})$$

**Schema F** (tree induction). If $f$ and $g$ are unary functors, $z$ a term, $s$ a binary functor, and we have derivations of:

$$f(O) = z, \qquad f(\langle x_0, x_1\rangle) = s(\langle x_0, x_1\rangle,\ \langle f(x_0), f(x_1)\rangle),$$
$$g(O) = z, \qquad g(\langle x_0, x_1\rangle) = s(\langle x_0, x_1\rangle,\ \langle g(x_0), g(x_1)\rangle),$$

then $f(x_0) = g(x_0)$.

**Schema F** expresses the uniqueness of tree recursion: if $f$ and $g$ satisfy the same recurrence (same base $z$, same step $s$), they agree on all inputs. Restricted to unary trees $\overline{n}$, this gives ordinary mathematical induction.

### Consistency

The system is *consistent* if $\mathbf{t} = \mathbf{f}$ (i.e., $O = \langle O, O\rangle$) is not derivable.

## 3 Key components

### 3.1 Proof encoding (thFunTFor, Theorem 14)

thFunTFor is a unary functor (defined as $\mathrm{R}(O, \mathrm{thFunStep})$) that maps proof-encoding trees to the code of the equation they prove. The step function thFunStep is a 26-branch dispatch handling all derivation rules.

*Theorem 14* (thm14E): if $H \vdash t = u$, then there exists a proof tree $\langle p_a, p_b\rangle$ such that

$$H \vdash \ \mathrm{thFunTFor}(\langle \hat{p}_a, \hat{p}_b\rangle) \ = \ \widehat{\ulcorner t = u \urcorner},$$

where $\ulcorner t = u \urcorner$ is the tree encoding of the equation. This is proved by induction on the derivation (1402 lines in `Thm14E.agda`).

### 3.2 Coded substitution (substTFor)

substTFor is a unary functor implementing coded substitution: given a coded term $\ulcorner t \urcorner$, it replaces every occurrence of variable $x_n$ by a replacement term $r$.

These two parameters ($r$ and $n$) are represented by free variables in substTFor. To avoid collision with the variables $x_0, x_1$ used in **Schema F** and in the template equation, the formalization places them at high indices:

- $x_{11}$ carries the replacement $r$,
- $x_{12}$ carries the target index $\overline{n}$.

Closing these variables gives the fully applied version:

$$\mathrm{closedSubstTFor}(r, \overline{n}) \ = \ \mathrm{substTFor}[r/x_{11},\ \overline{n}/x_{12}].$$

Correctness: for any term $t$, $\mathrm{closedSubstTFor}(r, \overline{n})$ applied to $\widehat{\ulcorner t \urcorner}$ equals $\widehat{\ulcorner t[r/x_n] \urcorner}$.

### 3.3 $\mathrm{Eq}(t,t) = \mathbf{t}$ via Schema F

$\mathrm{Eq}(t,t) = \mathbf{t}$ for all $t$ is proved using **Schema F**. Define $f = \mathrm{Eq}[\mathrm{I},\mathrm{I}]$ (so $f(t) = \mathrm{Eq}(t,t)$) and $g = \mathrm{K}_{\mathbf{t}}$. Both satisfy the same recursion:

- $f$: by the computation axioms for Eq and the recursive structure of Eq on $\langle t_1, t_2\rangle$;
- $g$: by $\mathrm{K}_{\mathbf{t}}$; the step reduces to $\mathrm{If}(\mathbf{t}, \langle \mathbf{t}, \mathbf{f}\rangle) = \mathbf{t}$ since $g$ returns $\mathbf{t}$ on all inputs.

**Schema F** gives $f(t) = g(t)$, i.e., $\mathrm{Eq}(t,t) = \mathbf{t}$. This avoids any heavy metalevel computation.

## 4 The Gödel sentence and diagonal construction

### Template

The Gödel sentence is built from a *template equation* with several free variables:

- $x_0$: will receive a proof encoding (the "proof slot");
- $x_1$: will receive a coded term (the "diagonal slot");
- $x_{11}, x_{12}$: the replacement and target parameters of substTFor (closed later).

The template is:

$$\mathrm{templateEqn} \;=\; \big(\mathrm{Eq}(\mathrm{thFunTFor}(x_0),\ \mathrm{substTFor}(x_1)) = \mathbf{f}\big).$$

It says: "the equation encoded by the proof in $x_0$ is not equal to the result of substituting via $x_1$." Leaving $x_{11}, x_{12}$ free at this stage keeps the tree code of the template small.

### Diagonalization

Let $N = \ulcorner\mathrm{templateEqn}\urcorner$ be the tree encoding of the template. The Gödel sentence is obtained by substituting $\hat{N}$ (the numeral for $N$) into the diagonal slot:

$$G \;=\; \mathrm{templateEqn}[\hat{N}/x_1].$$

After this substitution, $G$ says (informally): "for any proof tree in $x_0$, the equation it encodes is not the result of substituting $N$ into the template" — i.e., "no proof tree proves $G$."

### Abstract definitions (cached)

The following are computed once in `SubstTForPrecomp.agda` and cached:

- $N$: tree code of the template
- $\hat{N}$: the numeral $\widehat{N}$
- $G = \mathrm{templateEqn}[\hat{N}/x_1]$
- $\ulcorner G\urcorner$: tree encoding of $G$
- $\mathrm{cstf} = \mathrm{closedSubstTFor}(\widehat{\hat{N}},\ \overline{1})$: the substitution functor closed with replacement = code of the template numeral and target = variable $x_1$

### Heavy computation (one-time)

`SubstTForPrecomp.agda` contains one step requiring ∼108 seconds (computed once, cached in `.agdai`): proving that after instantiating $x_{11}$ and $x_{12}$, the open substTFor becomes cstf. This requires Agda to traverse $\widehat{\hat{N}}$ (∼400K term nodes) with substitution acting as identity on the closed subterm.

**Remark 3.** *The mathematical argument does not require this traversal. A general "substitution is identity on closed terms" lemma would eliminate it.*

## 5 Proof of Gödel I

The proof in `GodelII.agda` (0.2 seconds, 168 lines) follows Rose's Theorem 18 [3]. We assume $H \vdash G$ and derive $H \vdash \mathbf{t} = \mathbf{f}$, contradicting consistency.

1. **Assume** $H \vdash G$.
2. **Theorem 14** gives a proof encoding $e$ with
$$H \vdash \text{thFunTFor}(e) = \widehat{\ulcorner G \urcorner}.$$
3. **diagFTarget** (precomputed): $H \vdash \text{cstf}(\hat{N}) = \widehat{\ulcorner G \urcorner}$.
4. **Chain** steps 2 and 3 through $\widehat{\ulcorner G \urcorner}$ as middle term (trans + sym):
$$H \vdash \text{thFunTFor}(e) = \text{cstf}(\hat{N}).$$
The term $\widehat{\ulcorner G \urcorner}$ appears only as the middle of a transitivity chain — it is never touched by inst.
5. **Instantiate** $G$ with $e$ (inst on $x_0$):
$$H \vdash \text{Eq}(\text{thFunTFor}(e),\ \text{substTFor}(\hat{N})) = \mathbf{f}.$$
6. **Replace** thFunTFor($e$) with cstf($\hat{N}$) using $\text{cong}_L$ and the chain from step 4:
$$H \vdash \text{Eq}(\text{cstf}(\hat{N}),\ \text{substTFor}(\hat{N})) = \mathbf{f}.$$
7. **Instantiate** $x_{11}$ and $x_{12}$ (inst, using the cached computation from Section 4): the open substTFor($\hat{N}$) becomes cstf($\hat{N}$):
$$H \vdash \text{Eq}(\text{cstf}(\hat{N}),\ \text{cstf}(\hat{N})) = \mathbf{f}.$$
8. **Reflexivity** ($\text{Eq}(t, t) = \mathbf{t}$ via **Schema F**):
$$H \vdash \text{Eq}(\text{cstf}(\hat{N}),\ \text{cstf}(\hat{N})) = \mathbf{t}.$$
9. **Contradiction**: by trans, $H \vdash \mathbf{t} = \mathbf{f}$, violating consistency.

### What would be needed for Gödel II

For the second incompleteness theorem, one would need the system to derive Con $\to G$ internally. This amounts to formalizing Rose's Theorems 15–17 [3] inside the object system: the proof encoding correctness (ProofE) and the diagonal construction would need to be carried out as equational derivations, not just as Agda metalevel proofs. Cook [1] points out a possible error in Rose's Theorem 16 (p. 134 of [3]): the function $f(s(x))$ is neither identically zero nor identically non-zero, which invalidates the argument at that step. The present formalization bypasses this difficulty by finding an alternative route to Gödel II that avoids the problematic lemma.

## 6 File structure

| File | Time | Role |
|---|---|---|
| Step.agda | $< 1$s | Derivation system |
| Term.agda | $< 1$s | Terms, functors, coding functions |
| ThFun.agda | $< 1$s | Metalevel proof evaluator |
| ThFunTFor.agda | $< 1$s | Object-level proof evaluator (26 branches) |
| Thm14E.agda | $< 1$s | Theorem 14 (1402 lines) |
| SubstTForCorrect.agda | $< 1$s | closedSubstTFor correctness |
| SubstTForPrecomp.agda | 108s | **Heavy**: precomputed abstractions |
| SubstTForPrecomp2.agda | $< 1$s | Assembly from opaque pieces |
| GodelII.agda | $< 1$s | Gödel I proof (168 lines) |

Total: ∼40 files, ∼5000 lines. The 108-second computation is cached and not repeated.

## 7 Correspondence with Rose

| Rose [3] | This formalization |
|---|---|
| $th(y)$ | thFunTFor |
| $se(x, y, z)$ / $st(x, y, z)$ | substTFor / closedSubstTFor |
| $nu(x)$ | implicit via $\hat{N}$ |
| $N$ (template code) | templateCode |
| Schema F | ruleF (uniqueness of recursion) |
| Theorem 10 | thm14E (ProofE) |
| Theorem 14 | thm14E |
| Theorems 15–16 | Cases of thm14E + $\text{cong}_L/\text{cong}_R$; see [1] on Thm 16 |
| Theorem 17 | diagFTarget |
| Theorem 18 | godelII (= Gödel I in our terminology) |

## 8 Connection with Cook's PV

Cook [1] suggests (Section 8) that a natural formalization of feasibly constructive proofs should take a *programming approach*, where "proving $f(x) = g(x)$ amounts to proving the equivalence of two programs." The Guard/Rose system realizes this vision literally: proving $f(x_0) = g(x_0)$ via **Schema F** *is* proving that two recursive programs (the functors $f$ and $g$) compute the same function. Every derivation in the system is a chain of equational rewrites on functor expressions — no quantifiers, no propositional connectives, only computation axioms and the uniqueness-of-recursion principle.

## References


[1] Stephen A. Cook. Feasibly constructive proofs and the propositional calculus. *Proceedings of the 7th Annual ACM Symposium on Theory of Computing*, pages 83–97, 1975.

[2] James R. Guard. Lecture notes in recursive arithmetic. Princeton University, 1963.

[3] H. E. Rose. On the consistency and undecidability of recursive arithmetic. *Zeitschrift für mathematische Logik und Grundlagen der Mathematik*, 7:124–135, 1961.